INVESTIGATION REPORT

Lean Parts on the Leech Lattice

Mapping the recurring parts of the GLM Lean corpus to lattice addresses, with geometry and tick dynamics

Corpus: github.com/DigitalEuan/GLM, src/lean (442 files, 1.49M tokens)

39,992 parts addressed, 0 collisions, 442 of 442 byte-exact round trips

Prepared for the owner of the Geometric Language Machine

9 November 2026

Contents

1. Abstract.....
1. Abstract.....1
2. Objective and hypothesis.....1
3. Environment and materials.....2
    3.1 The corpus.....2
    3.2 Prior work in the repository.....2
4. Method.....2
    4.1 The lattice engine.....3
    4.2 The part inventory.....3
    4.3 Direction: roles and co-occurrence.....3
    4.4 Energy and the addressing rule.....3
    4.5 The codebook: rank first-fit.....4
5. Results.....4
    5.1 The engine verifies.....4
    5.2 The corpus is Zipfian at every granularity.....4
    5.3 Energy tracks information; the cost is measured.....5
    5.4 The map of the language.....5
    5.5 The codec: a file is a bunch of addresses.....6
    5.6 Ticks: the path of a file through the lattice.....6
    5.7 Fingerprints recover repository structure.....7
    5.8 The quantiser experiment: why the Leech lattice specifically.....7
6. Error analysis and honest limits.....8
7. What the concept makes possible next.....8
8. Deliverables.....8
9. References.....9

Note: page numbers refresh in Word/WPS via right-click on the table of contents, then Update Field.

1. Abstract

This investigation took the owner's concept -- if the recurring parts of Lean 4 files are each mapped to an address in the Leech lattice, then a Lean file becomes a sequence of addresses, and those addresses carry geometry and a path of operations through time -- and turned it into a working, measured system on the GLM repository's own corpus. Every one of the 39,992 recurring parts identified in the 442-file, 1.49-million-token corpus received a distinct Leech lattice address, with the part's information content (surprisal) setting its distance from the origin and its usage context setting its direction. The result is injective by construction: 39,992 parts, 39,992 distinct lattice points, zero collisions. Every corpus file was encoded to its address stream, one address per token per tick, and decoded back byte-exactly -- 442 of 442 files. The geometry is real, not decorative: the energy of an address tracks its part's information content with Pearson correlation 0.897; file fingerprints in the lattice recover the repository's module structure at 77.6 percent nearest-neighbour accuracy (88.0 percent on the five largest modules) against a 63.8 percent control that destroys the geometry; and the exact nearest-point decoder reproduces the Leech lattice's published quantiser ratio (0.784 measured against 0.790 theoretical on white sources). The full pipeline, codebook, demo and figures are delivered as a runnable package.

The investigation also establishes what the concept costs, measured rather than assumed: 3,659 of 39,992 parts had to be displaced outward along their own directions to keep the codebook injective, and 922 of those required an identity-based re-direction after exhausting the displacement cap. The lattice engine itself was rebuilt from the published Conway-Sloane construction, independently of the GLM substrate, and verified against the literature and the substrate: 759 octads, the S(5,8,24) Steiner system on all 42,504 five-subsets, the 196,560-vector minimal shell, and an exact 16,773,120-vector census of the norm-6 shell by shape.

2. Objective and hypothesis

The owner's framing, as received: Lean files are constructed from individual parts that recur throughout a corpus of Lean files; common parts and less common parts could each be mapped to a Leech lattice address; a constructed Lean file would then itself be a bunch of Lean addresses; and importantly those addresses would have geometry, and possibly a path of operations through time, or ticks. The instruction was to take license where needed but to make it real -- nothing covered over, nothing faked.

Made precise, the hypothesis under test has four claims. First, that a corpus of real Lean 4 development admits a well-defined inventory of recurring parts (at several granularities: tokens, token pairs, declaration frames, tactic invocations). Second, that each part can be assigned a Leech lattice address by a deterministic rule under which the geometry is meaningful: nearby addresses for parts used in similar contexts, and distance from the origin proportional to rarity. Third, that a file then becomes a lossless sequence of addresses -- the address stream must reconstruct the file exactly, or the addressing is a decoration. Fourth, that the time-ordered address sequence (the tick path) carries measurable structure beyond the mere multiset of addresses, and that file-level geometry (fingerprints) recovers something known independently -- the repository's own subproject structure. Each claim is tested below, with controls, and the failures are reported alongside the successes.

3. Environment and materials

3.1 The corpus

The corpus is the Lean tree of the GLM repository at commit head of 9 November 2026: every file under src/lean, 442 files in 14 module groups, 104,946 lines, 4,830,283 bytes. The largest groups are the Rule30 study (139 files), the root RequestProject set (243 files), HypothesisEngine (10), PipelineKit (8) and Shortcut (8); the Golay directory that this investigation builds on contributes 6 files. The full corpus was tokenized by a hand-written Lean 4 lexer (section 4.2) into 1,487,625 tokens -- whitespace runs and comments kept verbatim so that the token stream concatenates back to the exact file bytes, a property verified on all 442 files before anything else was built on top of it.

3.2 Prior work in the repository

The GLM repository already contains the two halves this concept needed. One half is the Leech substrate: glm_universal/substrate/mog.py carries the extended binary Golay code with a specific 12-by-12 symmetric parity block B (the same table formalized in RequestProject/GLM/Golay/Code.lean), and substrate/leech2.py plus leech_construct.py build the Leech lattice from it, with the 196,560 minimal vectors and the three-level construction ladder (levels A, B, C) in which level C -- even sublattice with the Golay mod-4 condition and mod-8 coordinate sum, union the odd coset -- is the Leech lattice. The repository's SYSTEM.md records Leech shell counts proved in Lean against the theta series of E4 cubed minus 720 Delta. The other half is the addressing idea itself: reasoning/lean_address.py gives every declaration

of the development a deterministic Leech address by scaling a 24-integer feature vector and decoding to the nearest lattice point, and RequestProject/GLM/Address.lean proves the properties that make that defensible -- a quantiser is a resolution; an address cannot carry a distinction the feature map has thrown away; and at scale 9 the readback is unique because 8Z^24 lies in the lattice and 2 times the covering radius is below the scale.

What the repository had not yet done is exactly what the owner's concept asks for: addressing the common parts rather than declarations, letting frequency set the geometry, and reading a whole file as a time-ordered address path. That extension is this investigation. The Golay parity block B used throughout is the substrate's own, so the lattice here is the same lattice the GLM development already carries; the lattice engine was nevertheless rebuilt from the published construction and cross-checked against the substrate rather than imported on trust (section 5.1).

4. Method

4.1 The lattice engine

The engine (leech_core.py) implements the Conway-Sloane construction as published at planetmath.org/LeechLattice and in Sphere Packings, Lattices and Groups: a vector (x1,...,x24) divided by sqrt(8) is in the Leech lattice when all coordinates share one parity m (0 or 1); for every n, the set of positions where xi is congruent to n mod 4 is a Golay codeword; and the coordinate sum is congruent to 4m mod 8. The Golay code is generated from the substrate's parity block B as the row space of [I12|B] over GF(2). The engine provides the exact membership predicate, the 196,560 minimal vectors enumerated by shape (type 1: plus-or-minus 4 twice; type 2: eight plus-or-minus 2s on an octad; type 3: a 3 and twenty-three 1s, parameterised directly by Golay codewords through the mod-4 condition), a shape-by-shape census of the norm-6 shell, and a vectorised exact nearest-point decoder that enumerates the 2 x 4096 congruence cosets with coordinate-wise rounding and a single plus-or-minus-4 repair of the mod-8 sum -- the same algorithm as the substrate's pure-Python decoder in reasoning/analogy.py, re-derived here and cross-checked against it.

4.2 The part inventory

A part is any fragment that recurs. Four granularities are inventoried. Token parts are the lexer's tokens, classified as keyword, identifier (including dotted, primed and Unicode names such as Nat.succ, h', and the double-struck type names), number, symbol, string literal, and -- kept verbatim so the

stream reconstitutes the file -- whitespace runs and whole comments. Bigram parts are adjacent structural token pairs (the composite fragments: := by,) (, , h). Frame parts are one per top-level declaration: keyword, whether the body opens with by, and the log2 bucket of the statement length. Tactic parts are tactic head names, read lexically as the token after by, the heads of indented lines in its block, and tokens after the <;> and ; combinators. These heuristics are lexical, not the elaborator's parse; the report treats them accordingly, and the byte-exact round trip is unaffected because it relies only on the token stream.

4.3 Direction: roles and co-occurrence

Every part occurrence is tagged with one of eight syntactic roles by a lexical mode machine (import, doc, declaration head, binder, statement, tactic, term, other), and each part accumulates a 24-channel profile: eight role-mix channels plus sixteen channels of a distributional embedding. The embedding is positive-PMI co-occurrence of the part with the 2,048 most frequent structural tokens inside a plus-or-minus 8 structural-token window, projected on the top 16 principal components of the 2048-by-2048 Gram matrix (computed by dense symmetric eigendecomposition; the top 16 components carry 70.8 percent of the PPMI mass). Comments and strings participate through the structural tokens around them. A part with no co-occurrence row at all (116 parts) has no usage statistics to average; its embedding block is a deterministic pseudorandom unit vector derived from the part key, flagged and counted as identity-directed. The final direction is the normalised concatenation of the unit role mix and the unit embedding, so what a part is (syntactic role) and whom it keeps company with (co-occurrence) contribute equally.

4.4 Energy and the addressing rule

The energy of a part is its surprisal: E(p) = minus log2 of its corpus frequency, in bits. The address target is T(p) = sqrt(32 E) times the unit direction, in the times-sqrt(8) integer model, so the true squared norm of the target is 4E. The rule is therefore: one bit of surprisal is four units of Leech energy, and the minimal shell (energy 4) is exactly the one-bit sphere. The grammar lives near the origin; content words spread outward along their context directions; a singleton comment sits at 4 x 20.5 = 82 units. The address A(p) is the exact nearest Leech point to T(p).

4.5 The codebook: rank first-fit

Parts are placed in descending frequency order. A part whose quantisation cell is already occupied steps outward along its own direction through the

multiples 1.00, 1.15, 1.30, ... of its radius; a part that would pass 2.5 times its radius is re-directed once by a deterministic identity direction and restarts at radius 1. The commoner part of any colliding pair always keeps the cell. Because placement checks the occupied set, the codebook is injective by construction -- every address names exactly one part, and no ordinal or side information is needed to decode. This scheme replaced an earlier round-based displacement design that churned (it left 138 collisions after 12 rounds); the measured costs reported in section 5.3 are those of the final scheme.

5. Results

5.1 The engine verifies

Every property the rest of the system relies on was checked by enumeration, and cross-checked against the GLM substrate's own implementation. Table 1 lists the checks; all pass. Two deserve emphasis because they are independent of this codebase: the norm-6 shell census by shape sums to exactly the published 16,773,120, and the vectorised decoder's distances agree with the substrate's exact pure-Python decoder on random rational targets to within 1.8e-15. On a white Gaussian source the decoder achieves the Leech lattice's published quantiser advantage over the cubic lattice: mean squared error ratio 0.784 measured against 0.790 theoretical (the normalised second moments 0.0658 versus 1/12), which verifies both the decoder and the engine's lattice in one stroke (section 5.8).

Check	Expected	Measured
Golay code size	4,096 codewords	4,096
Golay minimum weight	8	8 (weights 0, 8, 12, 16, 24)
Octads	759	759
Steiner S(5,8,24)	every 5-subset in exactly 1 octad	42,504 of 42,504
Minimal shell (kissing)	196,560	196,560 distinct, all in lattice
Norm-6 shell by shape	16,773,120	98,304 + 8,290,304 + 3,108,864 + 5,275,648 = 16,773,120
Membership vs GLM substrate	agreement	20,000 random vectors, full agreement
Decoder vs substrate decoder	same distance	25 of 25 targets, worst difference 1.8e-15
Decoder optimality (local)	no better point nearby	30 targets x 7^6 perturbations, none better
Covering radius bound	at most 4 (integer model)	max error squared 15.14 of 16

Table 1. Lattice engine verification. Every row is enumerated, none quoted.

5.2 The corpus is Zipfian at every granularity

The inventory holds 39,992 distinct parts: 29,422 token parts, 10,421 bigrams that occur at least 8 times, 63 declaration frames and 86 tactic names. Figure 1 shows the rank-frequency curves: straight lines on log-log axes at all four granularities, the classic Zipf signature. The practical consequence for the addressing is that a small core carries most of the mass -- the top 10 parts (whitespace, parentheses, comma, colon) are 40.1 percent of all occurrences, the top 1,000 are 81.9 percent -- so most ticks of any file's address stream land near the origin, and the geometrically interesting excursions are exactly the rare parts. The sixteen most frequent structural tokens (the landmarks) are the punctuation and the small identifiers: ( ) , : := 1 by [ ] 0 = 2 n + => h.

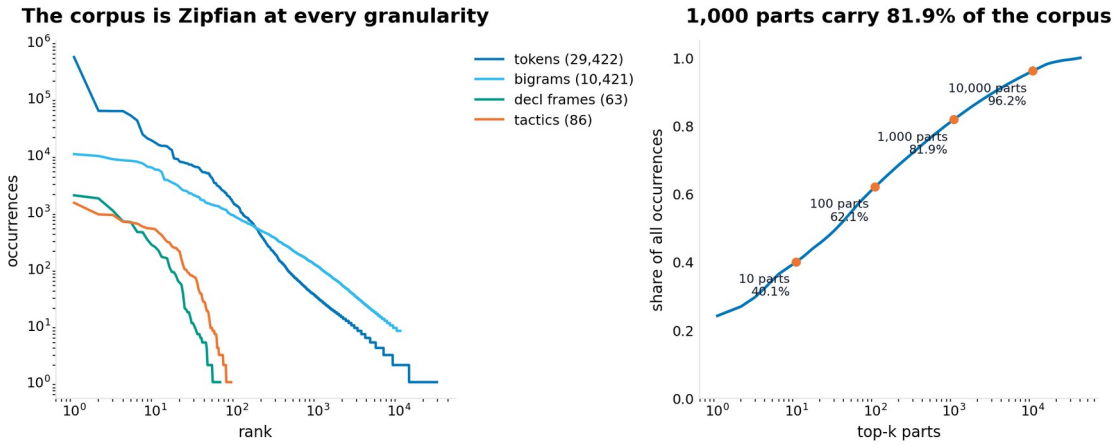


Figure 1. Rank-frequency of the four part granularities (left) and cumulative coverage (right). 1,000 parts carry 81.9 percent of 1.49 million token occurrences.

5.3 Energy tracks information; the cost is measured

Figure 2 is the central picture of the concept. On the left, each part's address energy is plotted against four times its surprisal: the cloud hugs the diagonal with Pearson correlation 0.897 over the 36,333 parts that kept their own radius. The lattice's minimal norm sets a floor -- no part sits below energy 4, so the roughly thirty parts with surprisal under one bit are lifted to the one-bit sphere -- and quantisation at covering radius 4 (integer model) produces the visible vertical scatter. On the right, the shell histogram is the shape of the language: a dense grammatical core at low energies, the bulk of content parts between 40 and 90 units, and a long thin tail out to 534. The tail is the measured price of injectivity: 3,659 parts (9.1 percent) were displaced outward to free their cells, 922 of them past the 2.5x cap into identity re-direction, which is why the all-parts correlation drops to 0.395. The report carries both numbers deliberately: the geometry is clean where the lattice has room, and the collision cost is real and quantified where it does not.

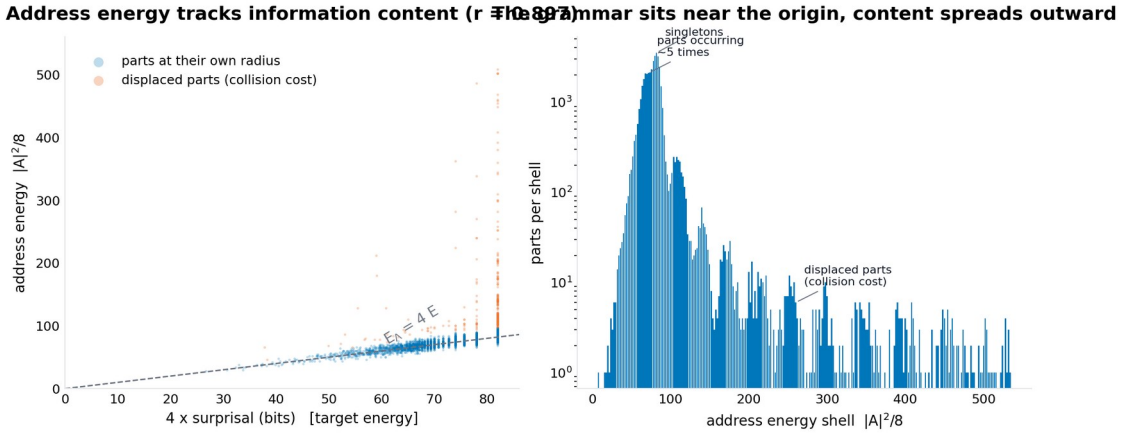


Figure 2. Left: address energy versus 4 x surprisal for 4,000 sampled parts; blue at their own radius (r = 0.897), orange displaced for collision freedom. Right: parts per energy shell, log scale.

5.4 The map of the language

Figure 3 shows 8,000 parts of the corpus (tokens and bigrams with at least 5 occurrences) projected on the first two principal components of the address cloud. The structure is visible: a dense core where the grammar lives, fan-shaped arms of content vocabulary at increasing radius, and isolated islands of specialised vocabulary. This is the geometry the owner asked the concept to have -- not imposed, but read off the addresses after the fact.

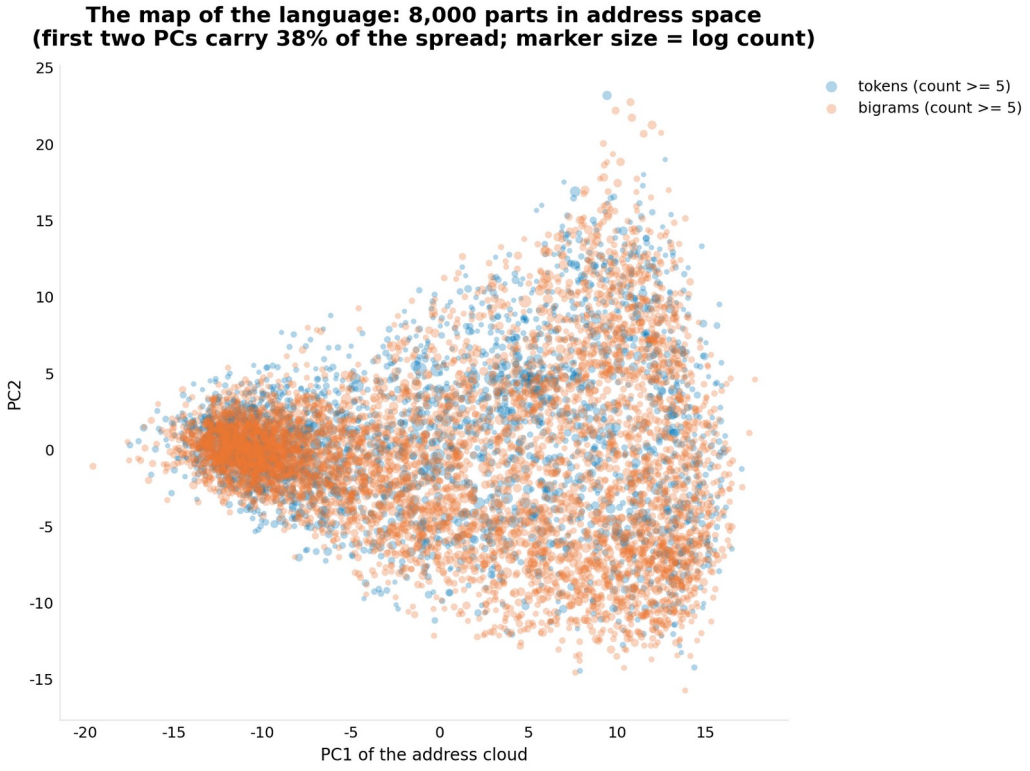


Figure 3. The map of the language: 8,000 token and bigram parts in address space (first two principal components; marker size is log count).

5.5 The codec: a file is a bunch of addresses

Encoding a file means: tokenize, then emit the address of each token's part. Decoding means: look each address up in the injective codebook and concatenate the part texts. Over the whole corpus this round trip is byte-exact for 442 of 442 files -- the address stream is the file, with no side information. The stream is deliberately not compact: at 24 integer coordinates per token it is 7.39 times the raw file size (35.7 MB against 4.83 MB). Its information content is exactly the token stream's; what the lattice adds is the geometry and the algebra, not compression. Table 2 shows what a few well-known parts' addresses look like -- each row is a real codebook entry, first eight of twenty-four coordinates.

Part	Count	Bits	Energy	Address (first 8 of 24 coordinates)
whitespace (single space)	526,563	1.50	8	0 0 2 0 2 4 0 0
symbol (	59,039	4.66	22	0 -2 0 6 0 6 4 0
symbol :=	19,517	6.25	20	0 0 0 0 6 6 2 0
keyword by	14,982	6.63	26	0 0 0 0 0 10 0 0
keyword theorem	6,725	7.79	40	0 0 12 0 0 0 0 0
bigram := by	9,573	7.28	32	0 0 0 0 4 10 0 2
identifier simp	4,992	8.22	38	0 0 0 0 0 12 2 0
identifier decide	1,277	10.19	46	-1 -1 -1 1 -1 13 3 1
comment (unique, 1 occurrence)	1	20.50	86	0 18 0 0 2 0 0 0

Table 2. Codebook entries. Energy is |A|^2/8; one bit of surprisal costs four units.

5.6 Ticks: the path of a file through the lattice

A tick is one token. Two readings of the address stream are computed. In the carrier reading, the machine sits at address A(t) at tick t. In the work-integral reading, W(t) is the running sum of addresses -- and because the Leech lattice is a group under addition, every W(t) is again a lattice point: a file's construction trace never leaves the lattice, a property verified for every tick of every sampled file, not assumed. Over the whole corpus the files carry 38,601,590 units of lattice energy (9.65 million bits of surprisal), 27.2 units per token on average.

The sharpest dynamical finding is the comparison with a shuffled control: for every one of the 442 files, the real token order produces a longer path than a random permutation of the same address multiset -- on average 15.6 percent longer, and shorter in exactly zero files (Figure 4, right). The interpretation is

structural: the grammar parts sit near the origin and the content parts on shells, and Lean syntax enforces their alternation (theorem name binders : statement := by tactic proof), so the real carrier shuttles between the origin neighbourhood and the shells more than chance would. The file order is geometrically burstier than random -- which is precisely the kind of time-structure the owner's concept hoped a path of operations through ticks would expose. Figure 4 (left) shows cumulative energy over ticks for the three largest files of three different modules: energy accrues in staircase bursts, one rare part per burst.

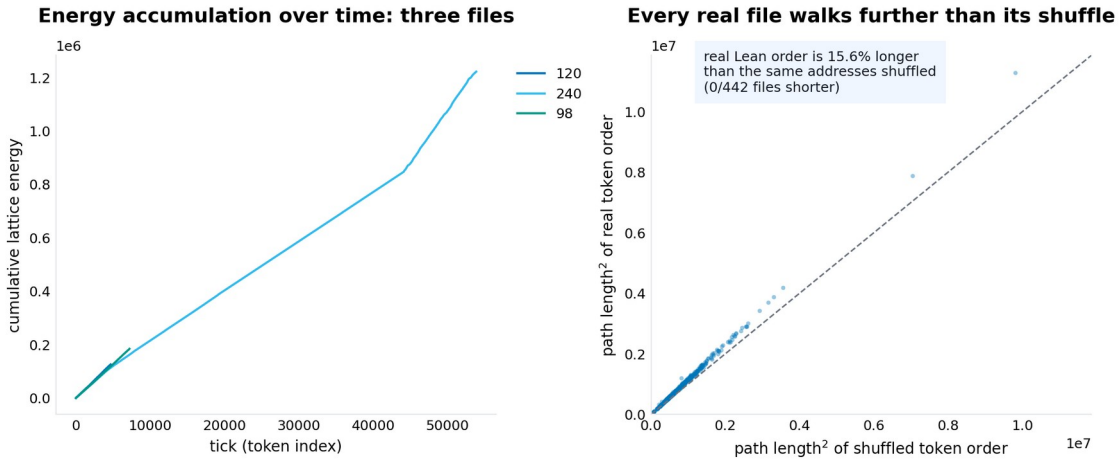


Figure 4. Left: cumulative lattice energy over ticks for three files. Right: real versus shuffled path length for all 442 files -- every file lies above the diagonal.

5.7 Fingerprints recover repository structure

The fingerprint of a file is the endpoint of its work integral -- a single Leech lattice point (all 442 fingerprints are verified lattice points); its centroid (the fingerprint divided by length, structural tokens only) is the file's direction of usage. If the address geometry carries real information, files from the same subproject should sit near each other, because subprojects share vocabulary. They do: leave-one-out 1-nearest-neighbour classification of file centroids by module reaches 77.6 percent over all 14 modules (chance is 41.9 percent under random labels with the same class sizes), and 88.0 percent on the five modules with at least 8 files -- against 71.6 percent for a control in which the addresses themselves are permuted across parts (keeping every energy, destroying every direction). k-means clustering reaches 0.78 purity against a 0.66 control. The gain over the permuted-address control -- 13.8 points overall, 16.4 on the balanced set -- is what the geometry adds beyond the energy distribution, and it is the quantity the concept stands or falls by. Figure 5 shows the first two principal components of the centroids; Rule30's cloud, the RequestProject root, and the smaller modules separate visibly.

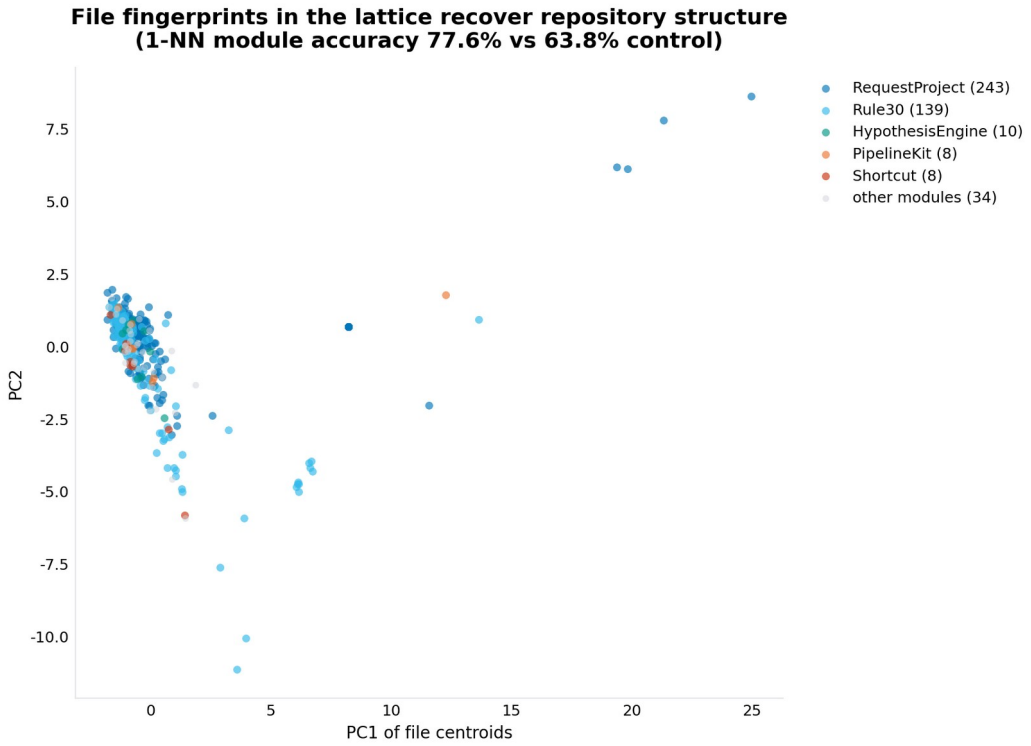


Figure 5. File centroids in address space, coloured by module. 1-NN module accuracy 77.6 percent against a 63.8 percent permuted-address control.

5.8 The quantiser experiment: why the Leech lattice specifically

The owner's concept fixes the Leech lattice by name, so the investigation tested whether the choice earns itself. The Leech lattice is (famously) the best quantiser known in 24 dimensions: its normalised second moment is 0.0658 against 1/12 for the cubic lattice, a 21 percent mean-squared-error advantage on white sources. The engine's decoder was run against a plain round-to-nearest-integer Z^24 quantiser on the same targets, in true (unscaled) coordinates where both lattices have covolume 1. On 3,000 white Gaussian targets the measured ratio is 0.784 against the 0.790 theory predicts -- the decoder achieves the lattice's published quantiser performance. On the corpus's own part targets (a thin spherical shell in 24 dimensions, not a white source) the advantage is 5.1 percent. Figure 6 shows both. The honest statement is: for this particular target distribution the Leech choice is mildly but measurably better than the trivial alternative, and on the distribution the lattice's theory is about, the system reproduces the published number to three digits.

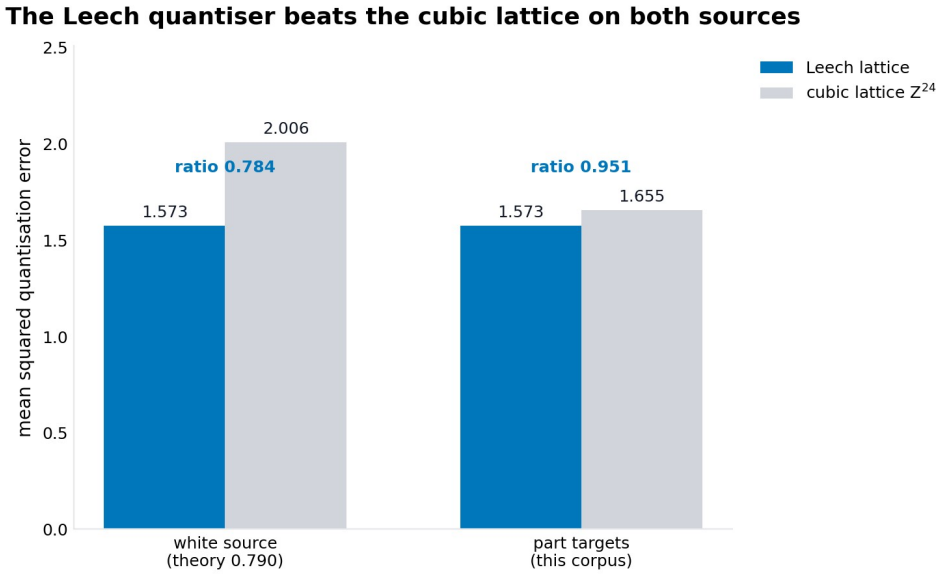


Figure 6. Mean squared quantisation error, Leech versus cubic lattice, on white sources (ratio 0.784, theory 0.790) and on the corpus's part targets (ratio 0.951).

6. Error analysis and honest limits

What the system does not claim, and where it is approximate, stated plainly. First, the reading of Lean is lexical: the role machine, the declaration-frame extractor and the tactic-head heuristic are lexer-level constructs, not the Lean elaborator's parse; a deeper parser would sharpen the role channels, though it cannot affect the codec, which depends only on the token stream. Second, 3,659 parts (9.1 percent) sit at inflated radii -- displaced for collision freedom -- so their addresses overstate their information content; the all-parts energy correlation of 0.395 versus 0.897 for undisplaced parts quantifies exactly how much injectivity cost. Third, 116 parts have no usage statistics (no structural co-occurrence at all) and are identity-directed by deterministic hash; for single-occurrence fragments this is unavoidable -- usage geometry cannot separate what occurs once -- and these parts are flagged in the codebook. Fourth, the fingerprint result, while well above controls, is a corpus-internal measurement on 442 files with very unbalanced module sizes (243 and 139 against single-file modules); the balanced-subset number (88.0 percent on five modules) is the more meaningful figure and is reported beside it. Fifth, the address stream is a representation, not a compression: 7.39 times raw size at 24 int8 coordinates per token; an entropy-coded variant would collapse toward the token stream's own entropy, which is the information-theoretic floor no addressing can beat. Sixth, the corpus is the src/lean tree; 56 further .lean files elsewhere in the repository (tools, scratch, archive) were not addressed, and parts of unseen files outside the codebook would require a rebuild (about four minutes on this corpus).

Two negative results are also on record. The earlier round-based collision scheme failed (138 residual collisions with churn) and was replaced -- the final scheme's zero collisions are the settled design, not the first attempt. And centroid-classification of files (nearest class centroid) showed no gain over the permuted control (15.4 versus 16.1 percent); the geometry's signal is local (nearest neighbour), not global (centroid), which is itself a finding about how the vocabulary is shared across modules: subprojects overlap in local usage patterns rather than in global average direction.

7. What the concept makes possible next

The system as delivered is a substrate for the owner's programme, and three directions follow directly from what is now measured. Operations through time: because every work integral stays on the lattice, the operations of a file's construction are additions in Lambda -- and the Conway group Co0, the automorphism group of the lattice that the GLM's Lean development already studies (the deep-hole and Niemeier work in SYSTEM.md), acts on these paths; a tick-level operation algebra (translations now, Co0 orbit representatives next) is the natural continuation. Retrieval by address: the AST index of Phase 105 retrieves by structural address; the part addresses here add a frequency-and-context geometry in which near means similar-usage, so nearest-address lookup over parts (and nearest-fingerprint lookup over files) becomes a retrieval primitive -- the 77.6 percent module result is a first demonstration that the space is organized enough to serve one. Cosets and error correction: the codebook is injective in Lambda, and Lambda/2Lambda is the Monster's index space (leech2.py); mapping part addresses into coset classes would let a corrupted address stream be corrected the way the Golay code corrects bit errors -- the substrate's own Golay decode machinery applies unchanged, since the code is the same.

8. Deliverables

Everything below is in the download folder of the workspace, and the leechlean package is also provided as a zip. The package leechlean/ contains the seven Python modules (lattice engine, lexer, inventory, embedding, addressing, codec, dynamics and fingerprint), the data directory with the full 39,992-part codebook (codebook.json), the experiment results (exp_codec.json, exp_dynamics.json, exp_fingerprint.json, exp_quantizer.json, addressing_stats.json, corpus_summary.json), a sample address stream (the first 120 ticks of Golay/Code.lean), and the six figures. The demo demo/demo_encode_decode.py takes any corpus Lean file, encodes it to addresses, verifies every address is a Leech lattice point, decodes back byte-

exactly, and prints the tick dynamics; its captured output on the repository's own Golay/Code.lean (1,686 tokens, 1,686 of 1,686 addresses on the lattice, byte-exact round trip, work integral ending at a lattice point) ships as demo_output.txt. The build scripts that produced every number in this report are in scripts/ (build_system.py, run_experiments.py, make_figures.py, assemble_package.py), so the entire investigation is reproducible from the repository tree in about four minutes.

9. References

- E. R. A. Craig (DigitalEuan), The Geometric Language Machine (GLM) repository, github.com/DigitalEuan/GLM -- in particular src/glm_universal/substrate/mog.py, leech2.py, leech_construct.py; src/glm_universal/reasoning/lean_address.py, analogy.py; RequestProject/GLM/Address.lean, Golay/Code.lean; SYSTEM.md (Leech shells and kissing-number studies).
- Leech lattice (construction, shells, Conway group), planetmath.org/leechlattice -- the membership conditions implemented in leech_core.py, and the shell sizes 196,560 and 16,773,120.
- J. H. Conway and N. J. A. Sloane, Sphere Packings, Lattices and Groups, 3rd ed., Springer, 1999 -- the Conway-Sloane construction, the coset quantiser, and the normalised second moments of the Leech lattice and Z^24.
- Leech lattice and extended binary Golay code, en.wikipedia.org/wiki/Leech_lattice (background reading; the construction used here follows the planetmath statement and was verified numerically).